

MongoDB Lab (M23BCS408B)

Lab 1:

Aim: Introduction to NoSQL and MongoDB 1a. Install MongoDB on your system (or use a pre-configured instance). Connect to the database using the MongoDB shell. Verify your connection by listing all the databases.

open command prompt

```
mongod --version
```

```
db version v7.0.8
```

```
mongosh
```

```
Using MongoDB: 8.0.4
```

```
Using Mongosh: 2.2.12
```

```
test> show dbs
```

```
admin 40.00 KiB
```

```
config 60.00 KiB
```

```
local 40.00 KiB
```

```
test 112.00 KiB
```

1b. Create a new database named 'my_database'. Switch to this database and create a collection named 'students'. Insert two sample documents into the 'students' collection, each representing a student with the following fields: 'name', 'age', 'city'.

```
test> use mydatabase
```

```
switched to db mydatabase
```

```
mydatabase>
```

Create the students Collection:

To create the students collection, use the createCollection() method.

This step is optional because MongoDB will automatically create the collection when you insert data into it,

but you can explicitly create it if needed:

```
mydatabase> db.createCollection("students")
```

```
>use test switched to db
```

```
{ "ok" : 1 }
```

```
mydatabase> >show collections
```

```
students
```

Insert operations

Insert a Single Document into students

```
mydatabase> db.students.insertOne({ name: "abhi",age: "20", city: "mysore"})
```

```
true:ok;
```

```
{ "_id" : ObjectId("5d7d3f28315728b4998f522f"),
```

Find All Documents

To retrieve all documents from the students collection:

```
mydatabase> db.students.find()
```

```
{ "_id" : ObjectId("5d7d3f28315728b4998f522f"), name: "abhi", age: "20", city: "mysore" }
```

2a. Add a new field 'email' to the existing student documents in the 'students' collection.

Update the documents

to include email addresses for each student.

```
my_database> use school
```

```
switched to db school
```

```
school> show collections
```

```
school>
db.students.insertMany([{name:"ram",age:"24",city:"mysore"},{name:"shyam",age:"32",city:
"mandya"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67b997ee04cb32daa9117b7f'),
    '1': ObjectId('67b997ee04cb32daa9117b80')
  }
}
school> db.students.updateMany({},{$set:{email:""}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
school> db.students.updateOne({name:"ram"},{$set:{email:"ram544@gmail.com"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
school> db.students.updateOne({name:"shyam"},{$set:{email:"shyam544@gmail.com"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
school> db.students.find()
[
  {
    _id: ObjectId('67b997ee04cb32daa9117b7f'),
    name: 'ram',
    age: '24',
    city: 'mysore',
    email: 'ram544@gmail.com'
  },
]
```

```
{
  _id: ObjectId('67b997ee04cb32daa9117b80'),
  name: 'shyam',
  age: '32',
  city: 'mandya',
  email: 'shyam544@gmail.com'
}
]
```

2b: Write a MongoDB query to retrieve all student documents with an age greater than 20. Additionally, write

another query to retrieve students whose city is 'Bengaluru'. Master CRUD operations

(Create, Read,

Update, Delete) on MongoDB document

```
school> db.students.find({age: {$gt:"20"}})
```

```
[
  {
    _id: ObjectId('67b997ee04cb32daa9117b7f'),
    name: 'ram',
    age: '24',
    city: 'mysore',
    email: 'ram544@gmail.com'
  },
  {
    _id: ObjectId('67b997ee04cb32daa9117b80'),
    name: 'shyam',
    age: '32',
    city: 'mandya',
    email: 'shyam544@gmail.com'
  }
]
```

```
school> db.students.updateOne({name:"ram"},{$set:{city:"banglore"}})
```

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

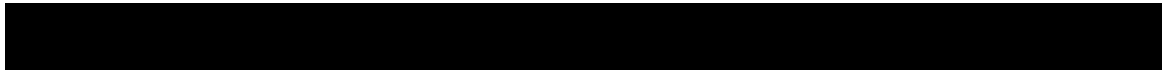
```
school> db.students.find({city:"banglore"})
```

```
[
  {
    _id: ObjectId('67b997ee04cb32daa9117b7f'),
    name: 'ram',
    age: '24',
    city: 'banglore',
    email: 'ram544@gmail.com'
  }
]
```

3a) Use the '\$regex' operator to retrieve all student documents whose name starts with the

letter 'A'. Then,
retrieve students whose city contains the word "Delhi".

```
db.Students.insertMany([{"name":"amar",age:"24",city:"banglore"}, {"name":"prathik",age:"25",  
city:"delhi"}])  
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('67c67f3546adc10b04117b7b'),  
    '1': ObjectId('67c67f3546adc10b04117b7c')  
  }  
}
```



```
test> db.Students.find({name:{$regex:"^A", $options:"i"}})
```

```
[
  {
    _id: ObjectId('67c67f3546adc10b04117b7b'),
    name: 'amar',
    age: '24',
    city: 'banglore'
  }
]
```

```
test> db.Students.find({city:{$regex:"delhi", $options:"i"}})
```

```
[
  {
    _id: ObjectId('67c67f3546adc10b04117b7c'),
    name: 'prathik',
    age: '25',
    city: 'delhi'
  }
]
```

3b) Create a new collection named 'books' and populate it with sample documents representing different books, each containing fields like 'title', 'author', and 'genre'. Write a query to retrieve all books that have the genre 'Science Fiction'

```
test> db.books.insertMany([
  {title:"dum",author:"Frank Herbert", genre:"Science Fiction"},
  {title:"Founda",author:"Isaac Asimov", genre:"Science Fiction"},
  {title:"1984",author:"George Orwell", genre:"Dystopian"},
  {title:"Brave New World ", author:"Aldas Huxley", genre:"Dystopian"},
  {title:"Neuromancer", author:"William Gibson", genre:"cyberpunk"}
])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67c683d246adc10b04117b7d'),
    '1': ObjectId('67c683d246adc10b04117b7e'),
    '2': ObjectId('67c683d246adc10b04117b7f'),
    '3': ObjectId('67c683d246adc10b04117b80'),
    '4': ObjectId('67c683d246adc10b04117b81')
  }
}
```

```
test> db.books.find({genre:"Science Fiction"})
```

```
[
  {
```

```
_id: ObjectId('67c683d246adc10b04117b7d'),  
title: 'dum',  
author: 'Frank Herbert',  
genre: 'Science Fiction'  
},  
{  
_id: ObjectId('67c683d246adc10b04117b7e'),  
tile: 'Founda',  
author: 'Isaac Asimov',  
genre: 'Science Fiction'  
}
```




```

]
test> db.books.find({genre:{$regex:"Science Fiction", $options:"i"}})
[
  {
    _id: ObjectId('67c683d246adc10b04117b7d'),
    title: 'dum',
    author: 'Frank Herbert',
    genre: 'Science Fiction'
  },
  {
    _id: ObjectId('67c683d246adc10b04117b7e'),
    title: 'Founda',
    author: 'Isaac Asimov',
    genre: 'Science Fiction'
  }
]

```

Aim: Aggregation and Data Analysis

Aggregation in MongoDB is a powerful framework that allows developers to perform complex data

transformations, computations and analysis on collections of documents. By utilizing the aggregation pipeline, users can efficiently group, filter, sort, reshape, and perform calculations on

data to generate meaningful insights.

\$count Calculates the quantity of documents in the given group.

\$max Displays the maximum value of a document's field in the collection.

\$min Displays the minimum value of a document's field in the collection.

\$avg

Displays the average value of a document's field in the collection.

\$sum Sums up the specified values of all documents in the collection.

\$push Adds extra values into the array of the resulting document.

\$match stage – filters those documents we need to work with, those that fit our needs

\$group stage – does the aggregation job

\$sort stage – sorts the resulting documents the way we require (ascending or descending)

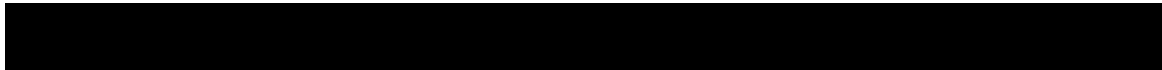
4a. Write an aggregation pipeline to calculate the average age of students in the 'students' collection.

```

db.Students.insertMany([
  { name: "Alice", age: 28, city: "New York" },
  { name: "Bob", age: 32, city: "Chicago" },
  { name: "Charlie", age: 45, city: "New York" },
  { name: "David", age: 39, city: "Los Angeles" },

```

```
{ name: "Eva", age: 27, city: "Chicago" },
{ name: "Frank", age: 51, city: "Los Angeles" }
])
db.Students.aggregate([
{
  $group: {
    _id: null,          // Group all documents together
    averageAge: { $avg: "$age" } // Calculate the average of the 'age' field
  }}]
  Output:
[ { _id: null, averageAge: 37 } ]
Example 2 when number is in string format like age:"25"
```



```

db.Students.aggregate([
  { $project: {
    age: { $toInt: "$age" } // Convert the 'age' field from string to integer
  }
}, {
  $group: { _id: null,          // Group all documents together
averageAge: { $avg: "$age" } // Calculate the average of the 'age' field
  }
}
])

```

Output:

```
[ { _id: null, averageAge: 37 } ]
```

4b. Use an aggregation pipeline to group students by city and count the number of students in each

city. Then, sort the results in descending order based on the count.

```

db.Students.aggregate([
  {
    $group: {
      _id: "$city",          // Group by the 'city' field
      count: { $sum: 1 }     // Count the number of people in each city
    }
  },
  {
    $sort: { count: -1 }     // Sort the results in descending order based on 'count'
  }
])

```

Output:

```
[
  { _id: 'Chicago', count: 2 },
  { _id: 'Los Angeles', count: 2 },
  { _id: 'New York', count: 2 }
]
```

Aim: Indexing and Performance Optimization

5a. Create an index on the 'age' field in the 'students' collection. Execute a query to retrieve students with an age greater than 25 and compare its execution time before and after creating the index.

Indexing in MongoDB

-

MongoDB uses indexing to make the query processing more efficient.

-

If there is no indexing, then MongoDB must scan every document in the collection and retrieve only those documents that match the query.

-

Indexes are special data structures that store some information related to the documents such that it becomes easy for MongoDB to find the right data file.

-

The indexes are ordered by the value of the field specified in the index.

```
db.Students.insertMany([
  { name: "Alice", age: 20, city: "New York" },
  { name: "Bob", age: 32, city: "Chicago" },
  { name: "Charlie", age: 45, city: "New York" },
  { name: "David", age: 39, city: "Los Angeles" },
  { name: "Eva", age: 22, city: "Chicago" },
  { name: "Frank", age: 51, city: "Los Angeles" }
])
```

```
])
```

Finding students with an age greater than 25

```
mydatabase> db.Students.find({age: {$gt:25}})
```

```
mydatabase> db.Students.find({age: {$gt:20}}).explain("executionStats")
```

output: Before the index

```
stage: 'COLLSCAN',
filter: { age: { '$gt': 25 } },
executionStats: {
  executionSuccess: true,
  nReturned: 6,
  executionTimeMillis: 5,
  totalKeysExamined: 0,
  totalDocsExamined: 6,
```

Create the index on the age field:

```
db.Students.createIndex({age:1})
```

age_1

To verify the created indexes, you can use the getIndexes method.

```
mydatabase> db.Students.getIndexes()
```

```
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { age: 1 }, name: 'age_1' }
]
```

```
// The default index on the _id field (_id_).
```

```
// An index on the age field (age_1),
```

```
>db.Students.find({age: {$gt:25}}).explain("executionStats")
```

output: After the index

```
stage: 'IXSCAN'
executionStats: {
  executionSuccess: true,
  nReturned: 6,
  executionTimeMillis: 14,
  totalKeysExamined: 6,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'FETCH'
```

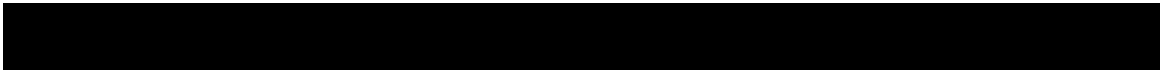
5b. Create a compound index on 'city' and 'age' fields. Run a query to retrieve students from 'Mysuru' who are older than 20 and analyze the improvement in performance compared to a query

without an index.

MongoDB Compound Indexes

- ❑ MongoDB Compound Indexes is an index on multiple fields in a MongoDB collection.
- ❑ It can be created to enhance the performance of queries that need to filter on multiple fields.
- ❑ A MongoDB compound index can cover queries that request only the indexed fields, improving efficiency by avoiding reading the actual documents.
- ❑ Compound Indexes do indexing on multiple fields of the document either in ascending or descending order i.e. it will sort the data of one field and then inside that it will sort the data of another field.

```
db.Students.find({ city: 'Mysuru', age: { $gt: 20 } })
```



Create the Compound Index:

```
db.students.createIndex({ city: 1, age: 1 })
```

```
city_1_age_1
```

```
db.Students.getIndexes()
```

```
[
```

```
  { v: 2, key: { _id: 1 }, name: '_id_' },
```

```
  { v: 2, key: { age: 1 }, name: 'age_1' },
```

```
  { v: 2, key: { city: 1, age: 1 }, name: 'city_1_age_1' }
```

```
]
```

```
db.Students.find({ city: 'Los Angeles', age: { $gt: 20 } }).explain("executionStats")
```

output:

```
executionStats: {
```

```
  executionSuccess: true,
```

```
  nReturned: 2,
```

```
  executionTimeMillis: 0,
```

```
  totalKeysExamined: 2,
```

```
  totalDocsExamined: 2,
```

To drop an index in MongoDB, you can use the `dropIndex()` method. The syntax for this method is as

follows:

```
db.collection.dropIndex(index_name)
```

```
db.students.dropIndex("city_1_age_1")
```

```
db.Students.find({ city: 'Los Angeles', age: { $gt: 20 } }).explain("executionStats")
```

```
executionStats: {
```

```
  executionSuccess: true,
```

```
  nReturned: 2,
```

```
  executionTimeMillis: 1,
```

```
  totalKeysExamined: 0,
```

```
  totalDocsExamined: 6,
```

Drop All Indexes Except the Default `_id` Index

```
db.students.dropIndexes()
```

Aim: Data Validation and Integrity

6a. Define a schema for the 'students' collection, specifying data types and required fields.

Add a custom

validation rule to ensure that the email field is a valid email format.

Schema validation is most useful for an established application where you have a good sense of how to

organize your data. You can use schema validation in the following scenarios:

☐

For a users collection, ensure that the password field is only stored as a string. This validation prevents users from saving their password as an unexpected data type, like an image.

❏

For a sales collection, ensure that the item field belongs to a list of items that your store sells. This validation prevents a user from accidentally misspelling an item name when entering sales data.

❏

For a students collection, ensure that the gpa field is always a positive number. This validation prevents errors during data entry.

```
db.createCollection("student",{  
  validator:{
```



```

    $jsonSchema:{           // what type of schema
      bsonType: "object",    //type of schema
      required:[ "name","email","gender"], // required filed in collection
    properties:{
      name:{
        bsonType:"string",
        description:"email is the required text field"},
      email:{
        bsonType:"string",
        description:"email is the required text field"},
        gender:{
          bsonType:"string",
          description:"gender is the required text field"}
    }
  }
}
})

```

6b. Attempt to insert a document with invalid data (e.g., an invalid email address or missing required fields)

and observe the error message. Explain the error message and how it relates to the defined schema and validation rules.

Example 1 : invalid email input

```

db.student.insertOne({
  name: "adhi",
  email: 123456, // not a sting type
  gender: "male"
})

```

Ouput:

MongoServerError: Document failed validation

Additional information: {

failingDocumentId: ObjectId('67efe8dc31b9cc8469fa4216'),

details: {

operatorName: '\$jsonSchema',

schemaRulesNotSatisfied: [

{

operatorName: 'properties',

propertiesNotSatisfied: [

{

propertyName: 'email',

description: 'email is the required text field',

details: [

```
{  
  operatorName: 'bsonType',  
  specifiedAs: { bsonType: 'string' },  
  reason: 'type did not match',  
}
```

```
consideredValue: 123456,  
consideredType: 'int'}}}}))
```

example 2: missing field gender

```
db.student.insertOne({  
  name: "adhi",  
  email: "adhirck@gmail.com "  
})
```

Output:

MongoServerError: Document failed validation

Additional information: {

failingDocumentId: ObjectId('67efe95531b9cc8469fa4217'),

details: {

operatorName: '\$jsonSchema',

schemaRulesNotSatisfied: [

{

operatorName: 'required',

specifiedAs: { required: ['name', 'email', 'gender'] },

missingProperties: ['gender']

}

]

}

}

Example 3: Correct schema

```
db.student.insertOne({  
  name: "adhi",  
  email: "adhirck1@gmail.com ",  
  gender: "male"  
})
```

}}

```
db.student.find()
```

```
{  
  _id: ObjectId('67efea4e31b9cc8469fa4218'),  
  name: 'adhi',  
  email: 'adhirck1@gmail.com ',  
  gender: 'male'  
}
```

Lab 7: Aim: Creating clusters using MongoDB Atlas

7a.Create a free MongoDB Atlas cluster and connect to it using the MongoDB Shell.

What is MongoDB Atlas?

MongoDB

Atlas is

a

fully

managed multi-cloud database.

It

handles

all

the

complexity

of deploying and managing on cloud service providers like AWS, Google Cloud Platform and Microsoft

Azure. Atlas is a good way to deploy, run and scale MongoDB in the cloud. Atlas offers a free tier (M0)

which makes it an excellent choice for small projects, testing, and learning MongoDB in the cloud without

incurring any cost.

What is MongoDB Shell?

The MongoDB Shell (also called mongosh) is a JavaScript-based interface that allows you to interact with

MongoDB databases from the command line. It enables data manipulation, querying, and performing

administrative tasks on the MongoDB database. MongoDB Shell is a powerful tool for developers working with MongoDB in local or cloud-based environments like MongoDB Atlas.

Step 1. Creating a Cluster in MongoDB Atlas

1. Sign Up for MongoDB Atlas

Go to the MongoDB Atlas website and sign up for an account, We can sign up using your email or directly with Google.

2. Build a Database

Now, go to the database option and click on the "Build a Database" button.

Step 2: Configure Your Cluster



Select the Free Tier Cluster (M0) option.



Choose your preferred cloud provider: AWS, Google Cloud, or Microsoft Azure.



Select the region where you want the cluster to be hosted (e.g., North Virginia, Frankfurt, etc.).



Rename the cluster if desired, then click "Create" to set up the free cluster.

Step 3: Set Up Access

Create a username and password for the MongoDB user and configure network access settings, allowing you to connect from any IP address. These credentials will be used to connect to MongoDB Atlas.

Step 4. Establishing a Connection to MongoDB Atlas from Mongo Shell

1. Connect Using Mongo Shell



In the MongoDB Atlas dashboard, click on the "Connect" button for your newly created cluster.



Choose "Connect with MongoDB Shell".



Select the appropriate Mongo Shell version (Mongosh) and follow the instructions.

Step 5: Run the Connection Command

Copy the connection string and run the command in our terminal, Then we can enter the password that we have set earlier with the username and password

Step 6: Writing Queries in Mongo Shell

1. Show Databases

This command lists all the databases in your MongoDB cluster:

```
show dbs
```

2. Create and Switch to a Database

Create a new database (if it doesn't exist) and switch to it:

3. Insert a Document into a Collection

```
db.student.insertOne({"username": "arindam369", "college": "JU"})
```

Output:

After executing the queries in the Mongo Shell, open the Collections in the MongoDB Atlas Cluster. We get the output given below.

Aim: User Management and Access Control

8a. Create a new user with read-only access to the 'students' database. Try to perform write operations using

this user and analyse the results. 8b. Create a role with read and write access to the 'books' collection and

assign this role to a new user. Verify that the user has the expected permissions.

Step 3: Set Up Access

Create a username and password for the MongoDB user and configure network access settings, allowing

you to connect from any IP address. These credentials will be used to connect to MongoDB Atlas.

Step 4. Establishing a Connection to MongoDB Atlas from Mongo Shell

1. Connect Using Mongo Shell

?

In the MongoDB Atlas dashboard, click on the "Connect" button for your newly created cluster.

?

Choose "Connect with MongoDB Shell".

?

Select the appropriate Mongo Shell version (Mongosh) and follow the instructions.

Step 7 : click database access then add new database user then enter username and password. And select role of the user. Read any database. Then select ok. New user is created.

1. Connect to MongoDB (as admin)

Mongosh

Use the admin database

use admin

Create a Custom Role with Read/Write on books collection

```
db.createRole({  
  role: "bookRWRole",  
  privileges: [  
    {  
      resource: { db: "library", collection: "books" },  
      actions: ["find", "insert", "update", "remove"]
```

```

    }
  ],
  roles: []
})
Expected Output:
db.createUser({
  user: "bookUser",
  pwd: "bookPass123",
  roles: [{ role: "bookRWRole", db: "admin" }]
})
Expected Output:
Successfully added user: {
  "user" : "bookUser",
  "roles" : [
    {
      "role" : "bookRWRole",
      "db" : "admin"
    }
  ]
}
Switch to New User
mongosh -u bookUser -p bookPass123 --authenticationDatabase admin
Use the library database
use library
Try Performing Write Operation
db.books.insertOne({ title: "MongoDB Basics", author: "John Doe" })
Expected Output:
{
  acknowledged: true,
  insertedId: ObjectId("...")
}
Try Reading
db.books.find()
Expected Output:
[
  { _id: ObjectId("..."), title: "MongoDB Basics", author: "John Doe" }
]
Try Writing to Unauthorized Collection (Optional Test)
db.otherCollection.insertOne({ test: 1 })
Expected Output
{
  ok: 0,
  errmsg: 'not authorized on library to execute command',

```

```
code: 13,  
codeName: 'Unauthorized'  
}
```

Lab 9: Aim: MongoDB and Python

9a. Using the 'pymongo' library, connect to a local MongoDB instance. Retrieve all documents from the

'students' collection and print them to the console

```
from pymongo import MongoClient
```

```
# Connect to local MongoDB server
```

```
client = MongoClient("mongodb://localhost:27017/")
```

```
# Access the 'students' database and 'student' collection
```

```
db = client['students']
```

```
collection = db['student']
```

```
# Retrieve all documents
all_students = collection.find()
# Print each document
print("All Students:")
for student in all_students:
    print(student)
```

9.b Write a Python script to insert a new document into the 'students' collection using the pymongo library.

Then, retrieve the newly inserted document and print it

```
from pymongo import MongoClient
# Connect to MongoDB (change host, port, and credentials if needed)
client = MongoClient("mongodb://localhost:27017/")
# Access the 'students' database and 'students' collection
db = client['students']
collection = db['students']
# Define the new document
new_student = {
    "name": "Alice Smith",
    "age": 20,
    "major": "Computer Science"
}
# Insert the document
insert_result = collection.insert_one(new_student)
print(f"Inserted document ID: {insert_result.inserted_id}")
# Retrieve the inserted document using the returned _id
retrieved_student = collection.find_one({"_id": insert_result.inserted_id})
# Print the retrieved document
print("Retrieved Document:")
print(retrieved_student)
```

10a. Setting up MongoDB Atlas Cluster and Configuring Security

Objective

To set up a MongoDB Atlas database cluster on a cloud provider (AWS, Azure, or GCP) and configure

basic security settings for controlled access. This step is foundational for integrating with

server less

applications or cloud services

1. Create a MongoDB Atlas Account and Cluster

a. Sign Up / Log In



Navigate to <https://www.mongodb.com/cloud/atlas>



Create an account using your email, Google, or GitHub.



After logging in, you'll be redirected to the Atlas Dashboard.

b. Create a New Project



Click on “New Project”



Give your project a name (e.g., StudentDBProject)



Click “Next”

c. Create a Cluster



Choose “Build a Cluster”



Select “Shared Cluster” (M0 Free Tier) – this is sufficient for learning and experimentation.



Name your cluster (e.g., Cluster0)



Proceed to cloud provider selection.

2. Choose a Cloud Provider and Region

a. Choose a Provider



Pick one of the following:

o AWS (default and recommended for beginners)



- o Azure
- o Google Cloud Platform (GCP)

b. Choose a Region



Choose a region closest to your location for better performance. Example:

- o Mumbai – if you are in India
- o US East (N. Virginia) – if you are in the US

c. Create Cluster



Click “Create Cluster”



Wait for a few minutes while the cluster is provisioned.

3. Configure Security Settings

A. Create a Database User

1. In the left panel, go to “Database Access”
2. Click “Add New Database User”
3. Enter:

- o Username: studentUser
- o Password: SecurePass123! (or generate securely)

4. Select Database User Privileges:

- o Role: Read and write to any database

5. Click “Add User”

B. Whitelist IP Address

1. Go to “Network Access”
2. Click “Add IP Address”
3. You have two options:
 - o To allow your current IP, click “Add Current IP Address”
 - o To allow access from any location temporarily (not secure), use: